

От чат-бота к агенту: агентная парадигма

ЛЕКЦИЯ 2 ИЗ 8 · ИНСТРУМЕНТЫ: ОТ ВЫЗОВА ФУНКЦИИ К АГЕНТУ

Генеративный искусственный интеллект и ИИ-агенты для транспортной логистики

РУТ (МИИТ) · 09.04.01 Информатика и вычислительная техника

Шесть вопросов на сегодня

#	ВОПРОС	БЛОК
1	Что такое инструмент	1
2	Какие инструменты бывают	2
3	Как вызов устроен технически и почему это вообще работает	3
4	Когда инструмент нужен и как его проектировать	4
5	Чем инструмент отличается от MCP-сервера	5
6	Как модель с инструментами становится агентом — и что такое агент	6

Потом демо и задание на практику.

Где мы остановились

Практика 1. Модель вернула JSON по вашей схеме: текст заявки → структура. Функция `parse_order_normalized()`.

Сегодня механика та же, направление другое. Модель снова вернёт JSON по вашей схеме — но это будет не ответ, а **просьба**: «вызови вот эту функцию с вот этими аргументами».

Разница между практикой 1 и практикой 2 — один шаг: **этот JSON надо выполнить и вернуть модели результат.**

Блок 1. Что такое инструмент

И почему без него модель выдумывает числа

Чего модель не умеет сама

Считать. Сумма по 170 тысячам строк — это не предсказание следующего токена. Ответ будет похож на правду и неверен.

Знать свежее и приватное. В обучающих данных нет ни вашей витрины, ни вчерашней выгрузки, ни номера вагона.

Действовать. Модель не пишет в базу, не отправляет сообщение, не запускает расчёт. Она порождает текст.

Schick et al., Toolformer, 2023: «модели... парадоксальным образом не справляются с базовыми вещами — арифметикой, поиском факта, — где гораздо более простые и маленькие модели превосходят их».

Как это выглядит на наших данных

«Сколько вагонов выгружено на дороге ЛСН в марте 2021?»

Без инструментов. Уверенный ответ с числом. Данных у модели нет — число выдумано целиком.

С инструментом. Модель просит `query_unload(2021-03-01, 2021-03-31, ЛСН)`, ваш код считает по витрине, модель отвечает: **9 761** вагон, период и фильтры названы.

Оба прогона увидите живьём в конце лекции. Разница между ними — не в модели. Модель одна и та же.

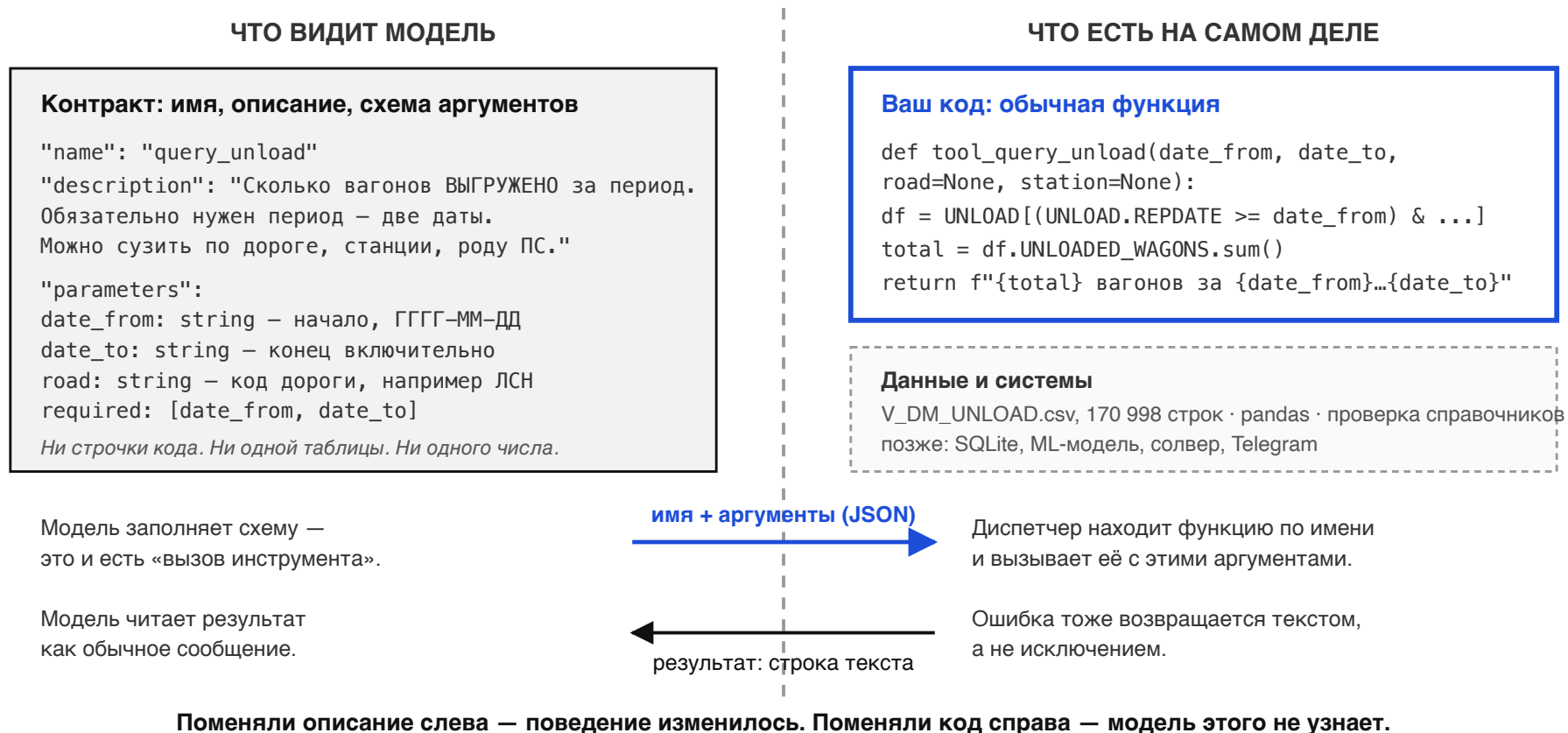
Определение

Инструмент — это функция, которую модель может попросить выполнить. У него две половины:

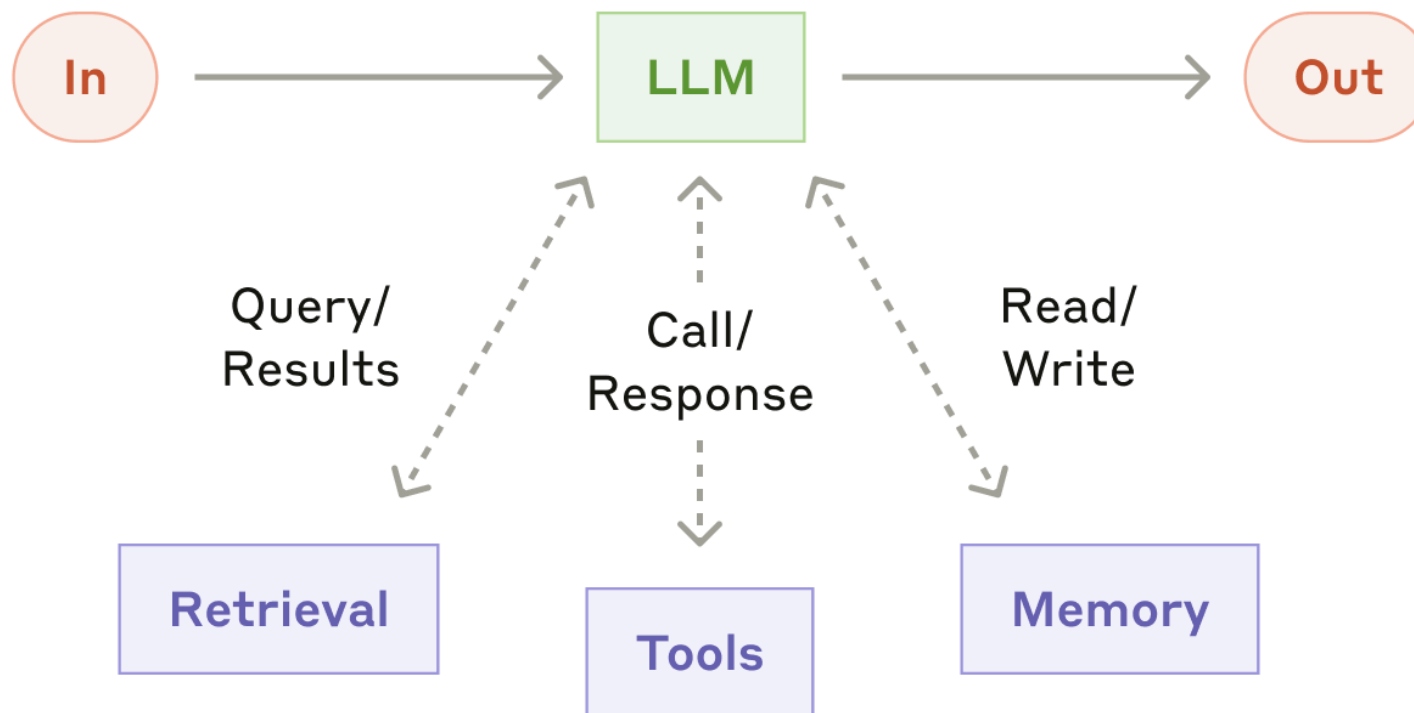
- **Контракт для модели:** имя, описание, JSON-схема аргументов. Больше модель не видит ничего.
- **Код для вас:** обычная функция, которая принимает эти аргументы и возвращает результат текстом.

Anthropic, «Writing effective tools for AI agents»: инструменты — «новый вид программ, отражающий контракт между детерминированными системами и недетерминированными агентами».

Что видит модель, а что есть на самом деле



Инструменты – одно из трёх расширений модели

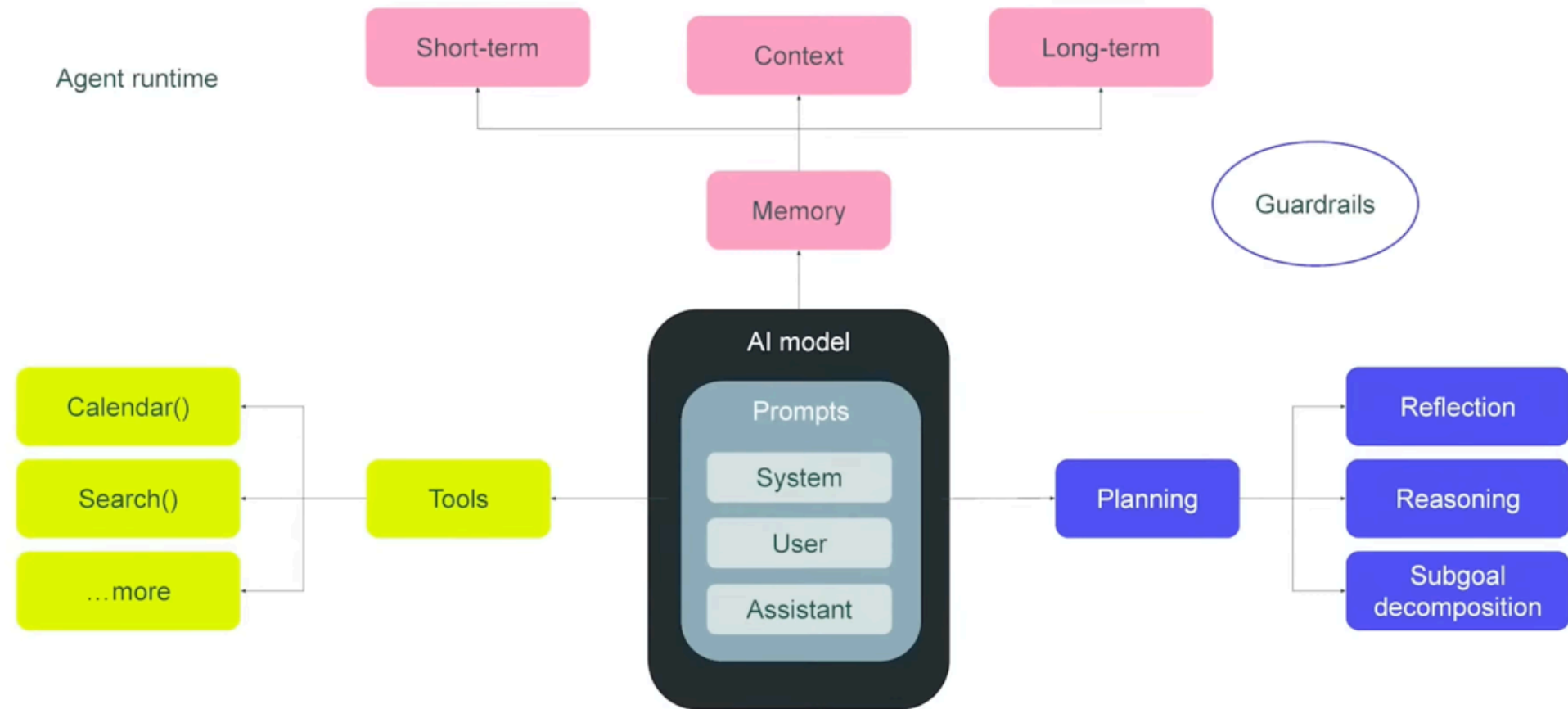


Базовый строительный блок агентных систем — модель, дополненная **поиском по данным, инструментами и памятью**. Сегодня — средняя стрелка. Память и поиск — на лекциях 3 и 4.

Anthropic, «Building effective agents», схема «The augmented LLM».

Agent OS

Agent = runtime(AI model + prompts + tools + memory + guardrails + planning skills)



Предыстория: модель, которая сама вставляет вызовы в текст

The New England Journal of Medicine is a registered trademark of [QA("Who is the publisher of The New England Journal of Medicine?") → Massachusetts Medical Society] the MMS.

Out of 1400 participants, 400 (or [Calculator(400 / 1400) → 0.29] 29%) passed the test.

The name derives from "la tortuga", the Spanish word for [MT("tortuga") → turtle] turtle.

The Brown Act is California's law [WikiSearch("Brown Act") → The Ralph M. Brown Act is an act of the California State Legislature that guarantees the public's right to attend and participate in meetings of local legislative bodies.] that requires legislative bodies, like city councils, to hold their meetings open to the public.

Toolformer, Meta AI, февраль 2023. Модель дообучили самой размечать, где в тексте позвать API — калькулятор, вопросно-ответную систему, перевод, поиск — и **вплетать результат** в дальнейший текст.

Вопрос 1

Для каких ваших задачи могут быть полезны подобны инструменты?

Блок 2. Какие инструменты бывают

Три способа разложить их по полкам

По тому, что инструмент делает

ТИП	ЧТО ДЕЛАЕТ	ПРИМЕР В КУРСЕ
Чтение	достаёт факты из данных: запрос к витрине, поиск документа	<code>query_unload</code> , <code>find_wagons</code> — сегодня
Вычисление	считает то, что модель не посчитает: прогноз, маршрут, формула	ML-модель на занятии 5, солвер на занятии 6
Действие	меняет мир: запись, отправка, запуск	фоновая задача и Telegram на занятии 7

Поиск по смыслу — частный случай чтения: аргумент не ключ, а вопрос.

Различие между чтением и действием — практическое: **чтение можно повторить, действие — нет.**

Карта инструментов курса



По тому, где выполняется код

	ИНСТРУМЕНТЫ КЛИЕНТА	ИНСТРУМЕНТЫ СЕРВЕРА
Кто выполняет	ваш код , в вашем приложении	инфраструктура провайдера модели
Что вы видите	запрос на вызов → выполняете → возвращаете	сразу результат, обработчика у вас нет
Примеры	ваши функции; вендорские схемы <code>bash</code> , <code>text_editor</code> , <code>computer use</code>	<code>web_search</code> , <code>web_fetch</code> , <code>code_execution</code>

Claude Docs, «Tool use with Claude»: client tools и server tools.

По тому, кто написал контракт

Свои. Схему и код пишете вы. Практика 2, сегодня.

Вендорские схемы. Провайдер публикует схему и обучает на ней модель, код выполняете вы: `bash`, `text_editor`, `memory`, `computer use`. Так устроены агенты-программисты.

Серверные. И схема, и код у провайдера: поиск в вебе, исполнение кода.

Чужие, по протоколу. Схему и код написал кто-то другой, вы только подключаете. Это MCP — блок 5.

То же самое глазами Google

Extensions — мост между агентом и внешним API, выполняется на стороне агента.

Functions — модель выдаёт имя функции и аргументы, **выполняет клиент**. Это наш случай.

Data stores — доступ к данным для поиска: векторная база, документы.

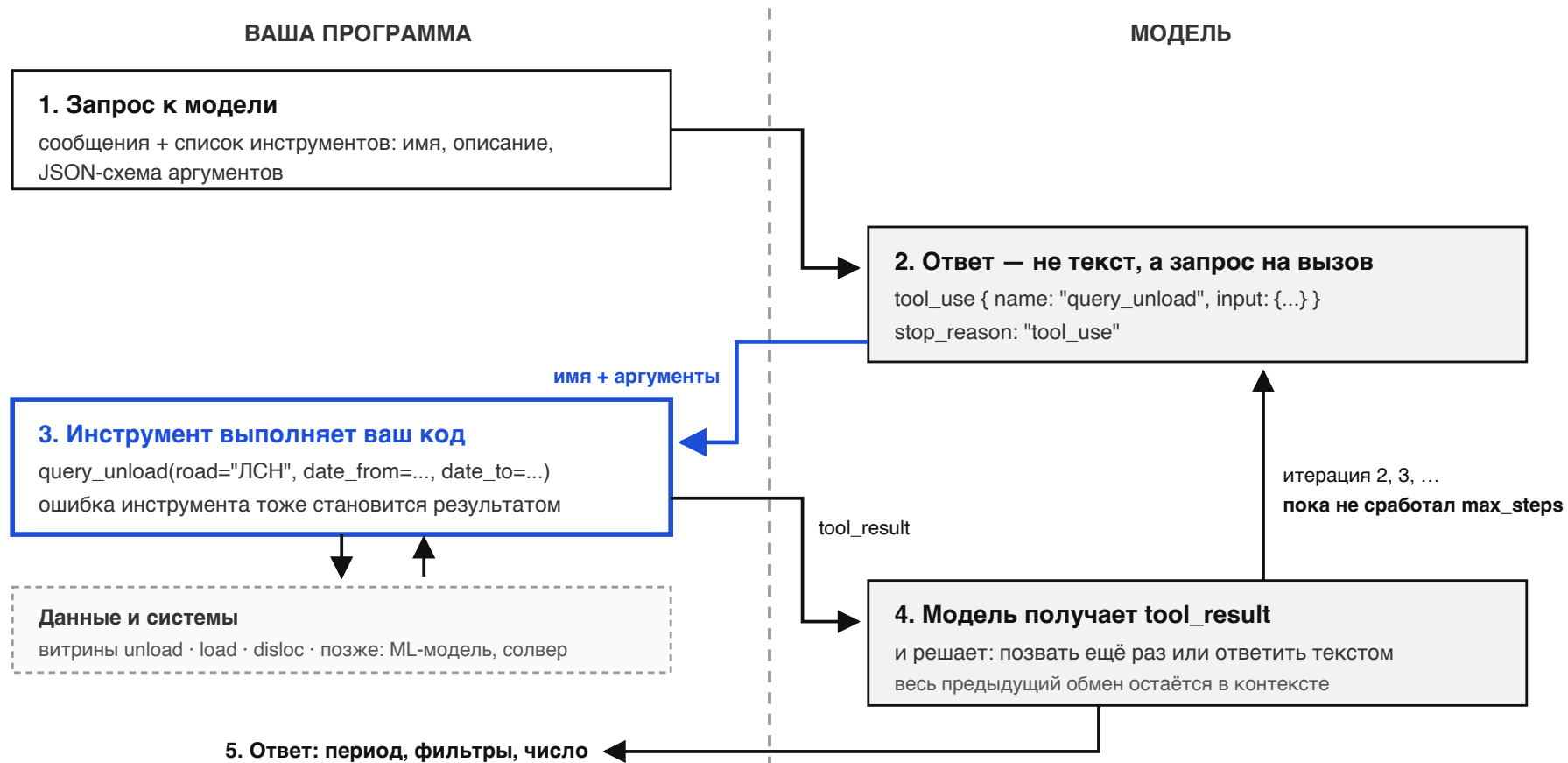
Три названия, одна и та же мысль: **модель выбирает, что позвать; выполняет кто-то другой**.

Wiesinger, Marlow, Vuskovic. Google, «Agents», сентябрь 2024, раздел «Tools».

Блок 3. Как устроен вызов

Технически — по полям. Идеино — почему это работает

Цикл целиком



Что происходит по протоколу

1. В запрос кладётся список инструментов: **имя, описание и JSON-схема аргументов**.
2. Модель отвечает не текстом, а **запросом на вызов**: имя инструмента и аргументы.
3. **Инструмент выполняет ваш код**. У модели нет доступа ни к базе, ни к сети.
4. Результат отправляется обратно **отдельным сообщением** — и цикл повторяется.
5. Цикл заканчивается, когда модель отвечает текстом вместо запроса на вызов.

Шаг 1: как выглядит инструмент в запросе

```
{
  "type": "function",
  "function": {
    "name": "query_unload",
    "description": "Сумма выгруженных вагонов за период. Фильтры: дорога, станция, род вагона.",
    "parameters": {
      "type": "object",
      "properties": {
        "date_from": {"type": "string", "description": "начало периода, ГГГГ-ММ-ДД"},
        "date_to": {"type": "string", "description": "конец периода, ГГГГ-ММ-ДД"},
        "road": {"type": "string", "description": "код дороги, например ЛСН"}
      },
      "required": ["date_from", "date_to"]
    }
  }
}
```

Это всё, что модель знает о вашей витрине. Больше ей ничего не покажут.

Шаги 2 и 4: запрос на вызов и ответ на него

```
// пришло от модели – это ещё не данные, это намерение
{"finish_reason": "tool_calls", "message": {"tool_calls": [
  {"id": "call_x7f2...", "type": "function", "function": {
    "name": "query_unload",
    "arguments": "{\\"date_from\\":\\"2021-03-01\\",...,\\"road\\":\\"ЛСН\\"}"}}]}}
```

```
// отправляете обратно вы, выполнив функцию сами
{"role": "tool", "tool_call_id": "call_x7f2...",
 "content": "{\\"metric\\":\\"выгружено вагонов\\",\\"wagons\\":9761,
  \\"period\\":[\\"2021-03-01\\",\\"2021-03-31\\"],
  \\"filters\\":{\\"road\\":\\"ЛСН\\"}}"} }
```

Аргументы приходят **строкой**, её ещё надо разобрать. А `tool_call_id` связывает вызов с результатом: потеряли его — сломали цикл.

Модель ничего не вызывает

Вызываете **вы**. Всё, что она умеет, — **определить, когда, что и с какими аргументами вызывать**.

Идейно: почему модель вообще умеет использовать инструменты

Вызов инструмента — способность модели, а не свойство API. Модель дообучена на том, чтобы по схеме выдавать структурный вызов вместо текста.

Отсюда следствие: модель, которая хорошо пишет текст, не обязана хорошо звать инструменты. Это разные навыки.

Схема — это часть промпта. Получив `tools`, API добавляет модели служебный системный промпт про вызов инструментов.

Claude Docs, «Tool use with Claude», раздел о ценообразовании: у Claude Opus 5 служебный промпт для инструментов — 286 токенов.

Кто решает, звать ли инструмент

TOOL_CHOICE	ПОВЕДЕНИЕ
auto	модель решает сама — по умолчанию
any / required	обязана позвать хотя бы один
конкретный инструмент	обязана позвать именно его
none	инструменты видны, звать нельзя

В режиме `auto` модель зовёт инструмент, когда **запрос ложится на описанную способность инструмента, а ответа ещё нет в контексте**. Это управляется и системным промптом: «сначала посмотри в данные» усиливает вызовы, «реши сам, нужен ли инструмент» — ослабляет.

В одном ответе может быть **несколько вызовов** сразу — если задачи независимы.

Схема вызова инструментов у разных провайдеров

	OPENAI-СОВМЕСТИМЫЕ: НАШ ПРОКСИ, DEEPSEEK	ANTHROPIC
Объявление	<code>tools : function.name , .description , .parameters</code>	<code>tools : name , description , input_schema</code>
Запрос на вызов	<code>message.tool_calls[] , finish_reason: "tool_calls"</code>	блок <code>tool_use</code> , <code>stop_reason: "tool_use"</code>
Возврат результата	сообщение <code>role: "tool"</code> с <code>tool_call_id</code>	блок <code>tool_result</code> с <code>tool_use_id</code>

Разбирали мы левый столбец — его вы и наберёте сегодня. У Anthropic поля названы иначе, схема та же: **имя, JSON-схема, результат назад**.

Четыре места, где цикл ломается

Нет ограничителя итераций. Модель зовёт инструмент, получает не то, зовёт снова. Без `max_steps` это бесконечный цикл за ваши деньги.

Ошибка инструмента улетает исключением. Правильно — вернуть текст ошибки **как результат**: у модели появляется ход на исправление.

Контекст растёт на каждой итерации. Каждый результат остаётся в переписке. Инструмент возвращает **ответ, а не дамп таблицы**.

Недостающий аргумент модель может додумать. Спросили «какая погода?» без города — модель подставит свой. Спросили «сколько выгрузили?» без периода — придумает период. Отсюда системный промпт с границами.

Блок 4. Когда и как использовать

Инструмент или промпт. И как его проектировать

Инструмент или промпт

В ПРОМПТ И КОНТЕКСТ	В ИНСТРУМЕНТ
стабильное знание: правила, термины, формат ответа	свежие данные: сегодняшняя дислокация
небольшой справочник, который влезает целиком	приватные данные: витрины, база, CRM
примеры того, как отвечать	всё, что надо посчитать точно
—	всё, что меняет мир

Правило простое: **если ответ можно записать в промпт раз и навсегда — это промпт. Если ответ зависит от того, что спросили, — инструмент.**

Не всё — агент с инструментами

СИТУАЦИЯ	ЧТО БРАТЬ
вход структурирован, шаги известны заранее	код без модели или цепочка вызовов: разбор → нормализация → запись
нужен текст-контекст для ответа: регламент, инструкция	поиск — найти и положить в промпт
нужно точное число или действие по запросу словами	инструмент
путь к ответу заранее не известен, но результат проверяем	цикл с инструментами — блок 6

Берите простейшее из того, что проходит проверку. Каждая автономная итерация — латентность, деньги и шанс, что ранняя ошибка утащит за собой всё.

Anthropic, «Building effective agents».

Инструмент — это интерфейс

- **Описание инструмента — это промпт.** Каждое слово в имени, описании и описаниях аргументов меняет поведение.
- **Лучше меньше инструментов, но толще.** Двадцать мелких функций модель путает.
- **Поиск вместо перечисления.** Не вываливайте в контекст всё, что есть.
- **Возвращайте то, что нужно для ответа.** Инструмент — не `SELECT *`.
- **Семантика источника — часть контракта.** Дислокация — срез, а не история. Заявки ГУ-12 — суточный снимок запаса: дни складывать нельзя, нужно среднесуточное. Не написали — модель сложит.
- **Встаньте на место модели.** У неё нет ничего, кроме имени, описания, схемы — и вашего результата.

Anthropic, «Writing effective tools for AI agents».

Одна строка описания решает исход

БЫЛО	СТАЛО
<code>query_load</code> — «погрузка»	<code>query_load</code> — «Количество погруженных вагонов за период. Фильтры: дорога, станция, род вагона. Не для выгрузки — для неё есть <code>query_unload</code> .»

Модель звала `query_load` там, где нужен был `query_unload`, — и отвечала уверенно и неверно.

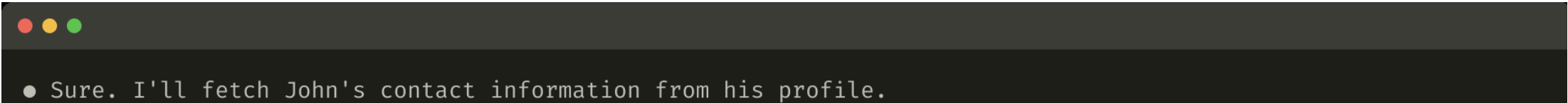
Починка заняла одну строку текста. **Кода не поменялось ни строчки.**

Ошибка – тоже результат. Вопрос, какой



```
• Sure. I'll fetch John's contact information from his profile.  
  
• asana - user_info (MCP)(userId: "john.doe@acme.corp")  
  L {  
    "error": {  
      "code": "RESOURCE_NOT_FOUND",  
      "status": 422,  
      "message": "Invalid value",  
      "details": {  
        "field": "userId",  
        "value": "john.doe@acme.corp",  
        "type": "invalid_value"  
      }  
    }  
  }  
}
```

Бесполезная: модели некуда двигаться.



```
• Sure. I'll fetch John's contact information from his profile.
```

Ответ инструмента — это контекст, за который вы платите

```
● I'll search slack for recent bug reports and use the detailed format to see which channel IDs and threads to investigate further.

● slack - search (MCP)(query: "bug", sort: "timestamp", sortDir: "desc", limit: 100, responseFormat: "detailed")
  L Search results for: "bug"

    ══ Result 1 of 89 ══
    Channel: #dev (C1234567890)
    From: @jane.doe (U123456789)
    Time: 2024-01-15 10:30:45 UTC
    TS: 1705316445.123456
    Text: Found a critical bug in the login flow.

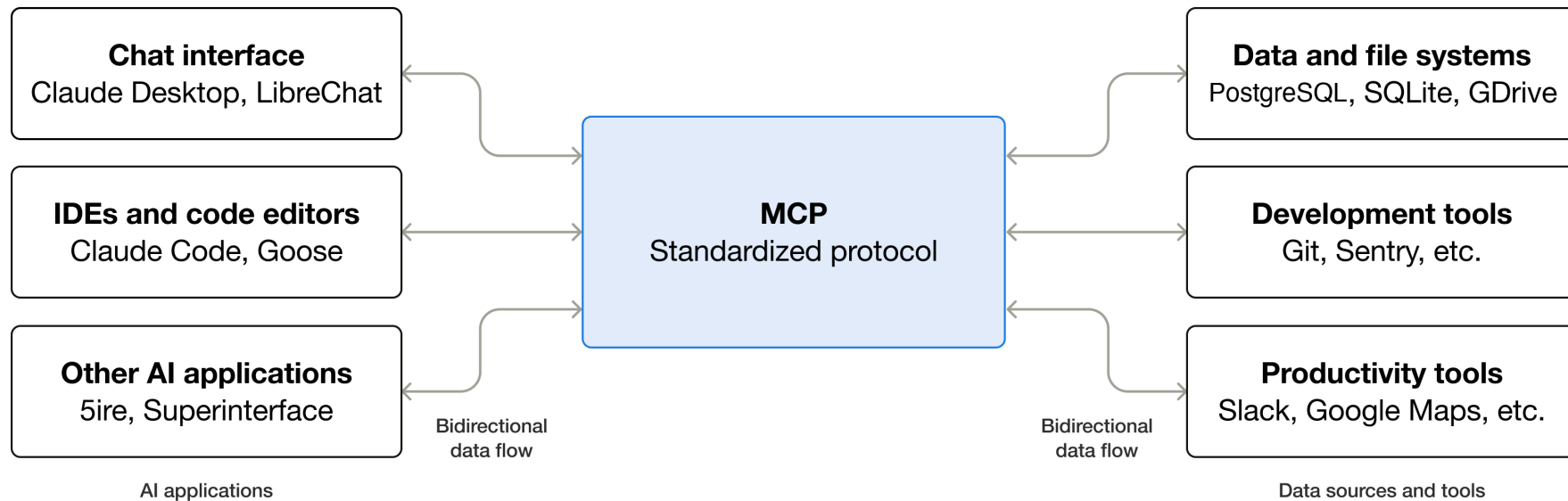
    ══ Result 2 of 89 ══
    Channel: DM with @john.smith
    From: @john.smith (U987654321)
    Time: 2024-01-14 15:22:18 UTC
    TS: 1705247738.234567
    Text: The bug report for issue #123 is ready for review
    Files: bug-report-123.pdf
    ...
```

Подробный ответ: 206 токенов, идентификаторы и служебные поля.

Блок 5. Инструменты и МСР

Чем инструмент отличается от МСР-сервера

Проблема, которую решает протокол



Без стандарта каждое приложение пишет свой адаптер к каждой системе: **M приложений × N источников**. MCP ставит между ними один разъём.

Model Context Protocol, документация: «USB-C для ИИ-приложений».

MCP — что это

Model Context Protocol — открытый стандарт подключения ИИ-приложений к внешним системам: данным, инструментам, шаблонам промптов. Открыт Anthropic в ноябре 2024, сегодня его поддерживают Claude, ChatGPT, VS Code, Cursor и другие.

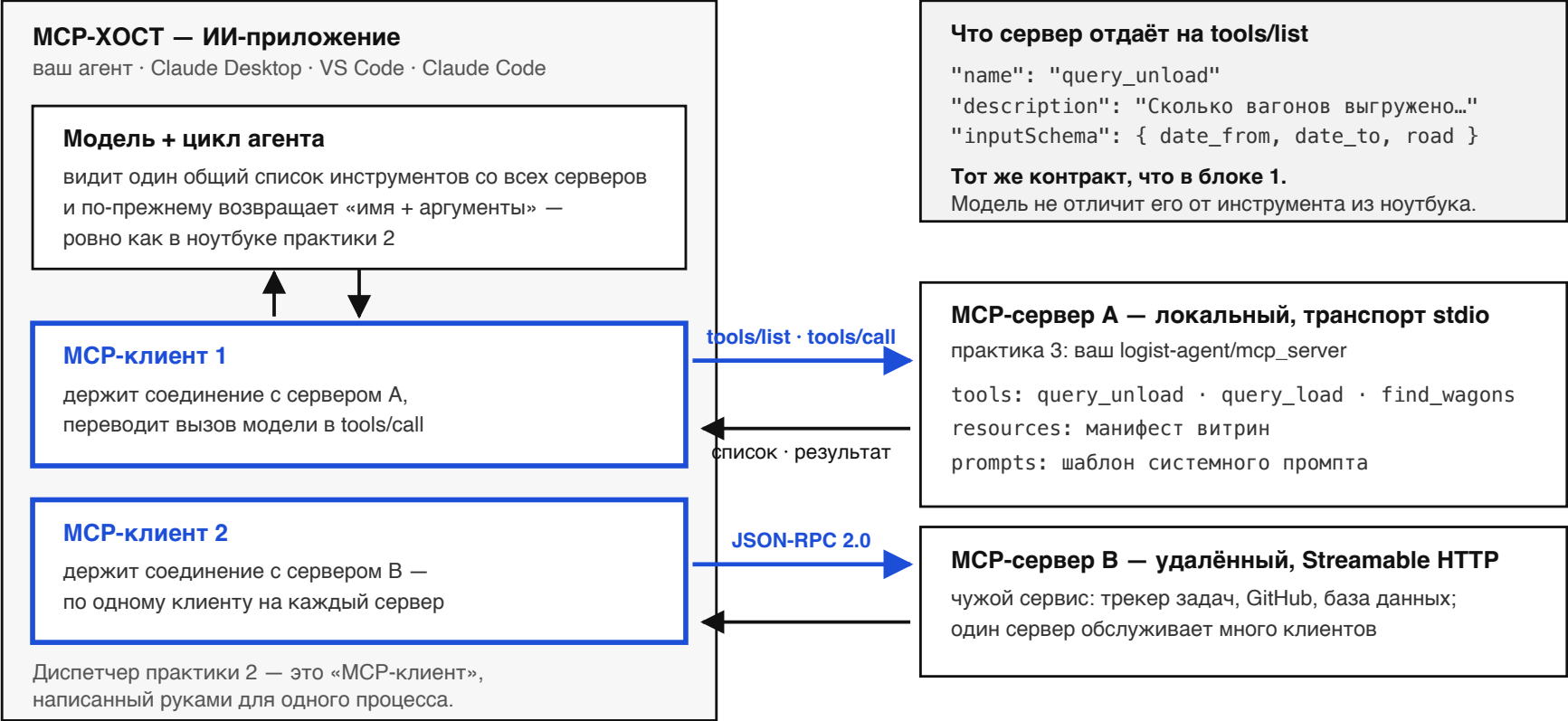
Что даёт сервер: три вида примитивов —

- **tools** — функции, которые приложение может вызвать;
- **resources** — данные для контекста: файл, запись, схема базы;
- **prompts** — шаблоны для типовых взаимодействий.

На чём построен: JSON-RPC 2.0 поверх stdio (локально) или Streamable HTTP (удалённо).

Model Context Protocol, «Architecture overview».

Хост, клиент, сервер



МСП не заменяет инструменты. Это способ их поставлять — любому хосту, без адаптера под каждый.

Два запроса, которые делают всё

```
// клиент → сервер: что у тебя есть?  
{"jsonrpc": "2.0", "id": 2, "method": "tools/list"}  
// сервер → клиент  
{"result": {"tools": [{"name": "query_unload",  
  "description": "Сколько вагонов выгружено за период...",  
  "inputSchema": {"type": "object", "properties": {"date_from": ...}}}]}}
```

```
// клиент → сервер: выполни  
{"jsonrpc": "2.0", "id": 3, "method": "tools/call",  
  "params": {"name": "query_unload", "arguments": {"date_from": "2021-03-01", ...}}}  
// сервер → клиент  
{"result": {"content": [{"type": "text", "text": "9 761 вагон"}]}}
```

Сравните с шагами 1, 2 и 4 из блока 3. **Те же три поля, та же строка результата.**

Формат — Model Context Protocol, «Architecture overview», пример с tools/list и tools/call.

Инструмент против MCP-сервера

	ИНСТРУМЕНТ	MCP-СЕРВЕР
Что это	функция со схемой внутри вашего процесса	отдельная программа , отдающая набор инструментов по протоколу
Кто вызывает	ваш диспетчер, по имени	MCP-клиент, через <code>tools/call</code>
Кто может подключить	только ваш код	любой хост : Claude Desktop, VS Code, ваш агент, чужой агент
Что видит модель	имя, описание, схема	ровно то же — из ответа <code>tools/list</code>
В курсе	практика 2, сегодня	практика 3: те же три инструмента уезжают за MCP. Лекция 5 — протокол целиком

MCP не заменяет инструменты. Это способ их поставлять. Для модели разницы нет.

Блок 6. От модели с инструментами — к агенту

Что такое агент и где он начинается

Один вызов — ещё не агент

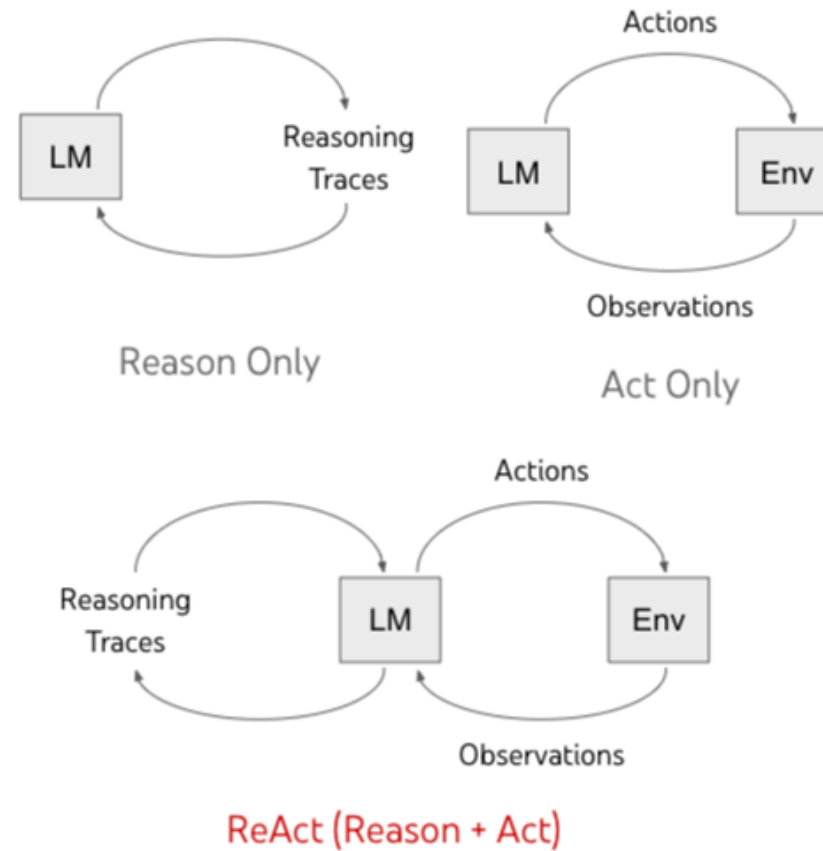
Запрос → **запрос на вызов** → выполнили → **результат инструмента** → ответ. Один круг.

Так работает «умный поиск» в чате, и агентом его не зовут.

Агент начинается там, где круг повторяется, и модель сама решает, повторять ли: позвать ещё раз, позвать другой инструмент, остановиться.

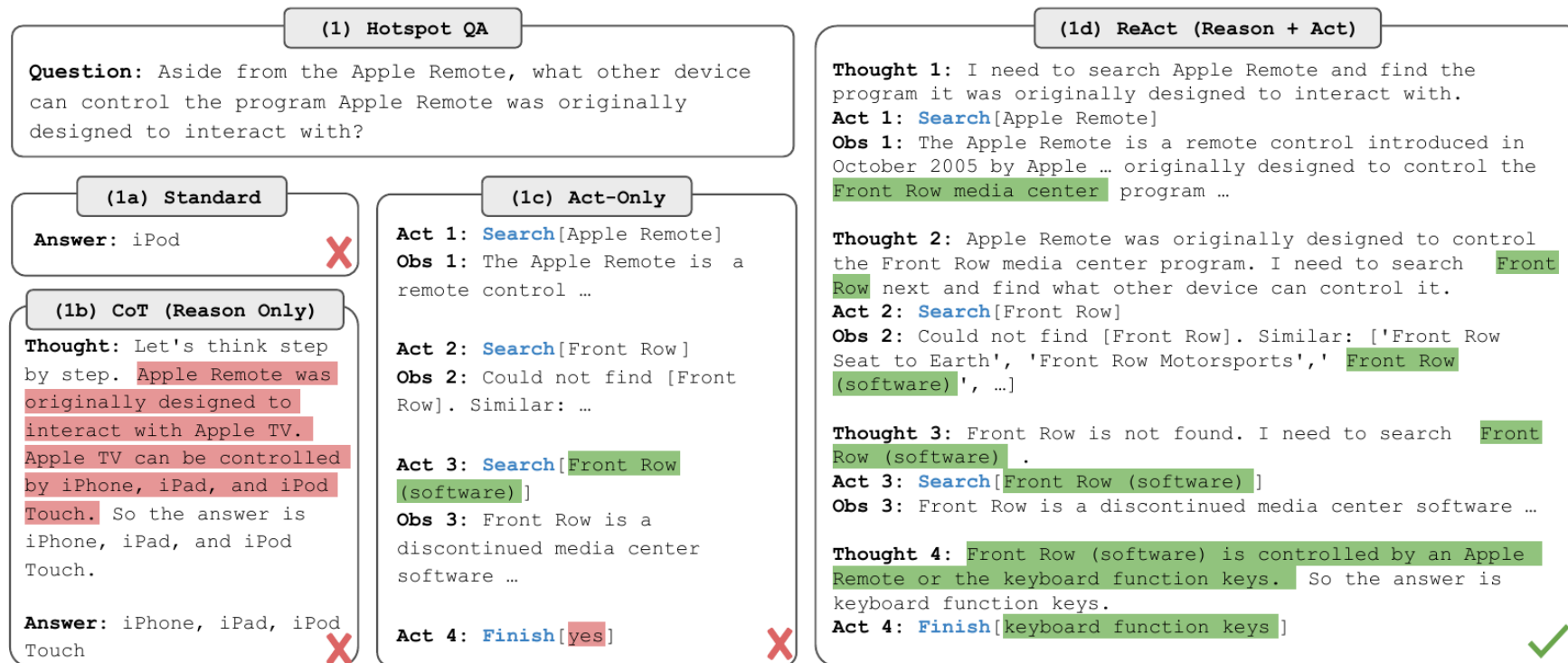
То есть агент — это **не модель и не набор инструментов**. Это **цикл вокруг модели**, в котором следующий шаг выбирает она.

ReAct, 2022: откуда взялся цикл



Yao et al., «ReAct: Synergizing Reasoning and Acting in Language Models», октябрь 2022, рис. 1.

ReAct, 2022: откуда взялся цикл



Только рассуждение (1b) галлюцинирует; только действия (1c) не понимают, что ищут.

Чередование мысли, действия и наблюдения доходит до ответа. На интерактивных задачах — **+34 %** и **+10 %** к прежним подходам.

В вашем цикле сегодня видны действие и наблюдение; мысль модель держит при себе.

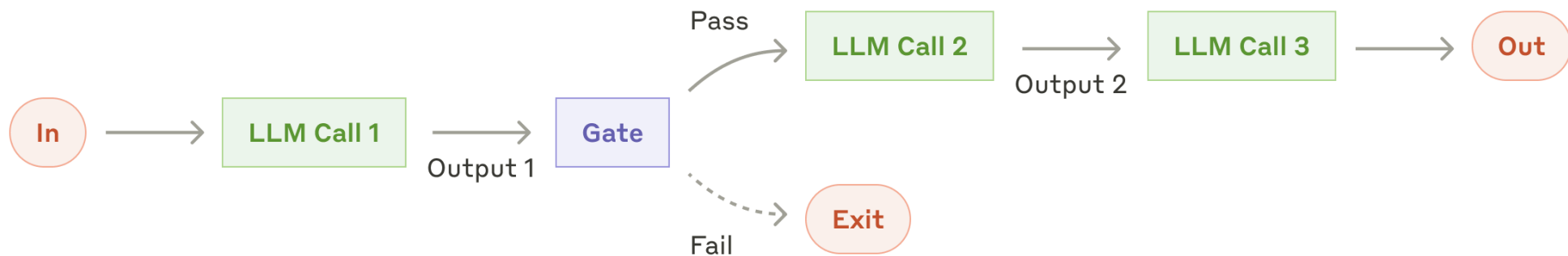
Два определения, одна граница

Google, «Agents». Агент — **приложение**, которое достигает цели, наблюдая за миром и действуя на него доступными инструментами. Три компонента: **модель, инструменты, слой оркестрации** — цикл, идущий до цели или до точки остановки.

Anthropic, «Building effective agents». **Workflow** — модель и инструменты оркеструются заранее написанными путями в коде. **Agent** — модель **сама направляет свой процесс** и выбор инструментов.

Граница проходит **не по «умности» модели**, а по тому, **где лежит решение о следующем шаге**: в вашем `if` или в модели.

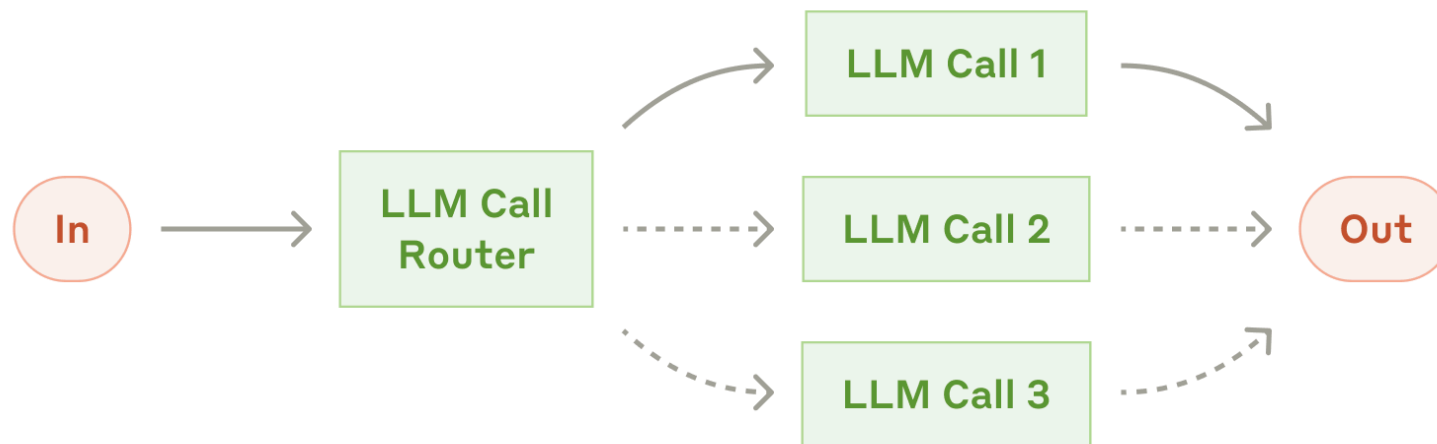
Путь задан кодом: цепочка



Шаги фиксированы, между ними проверка. Разбор заявки → нормализация → запись. Модель есть, агента нет.

Anthropic, «Building effective agents», схема «The prompt chaining workflow».

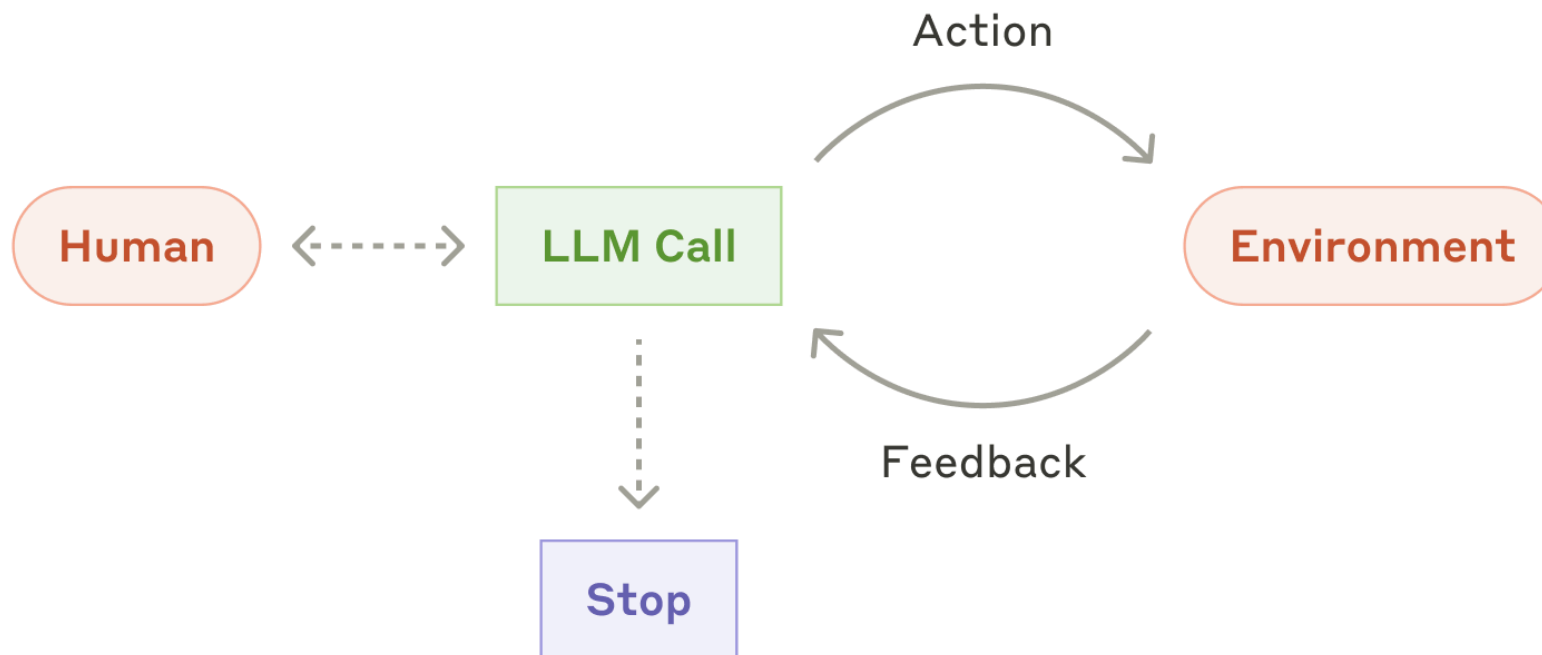
Путь выбирается из готовых веток: маршрутизация



Модель классифицирует вход и передаёт в свою ветку: «это про план или про факт?». Выбор есть, но из списка, который написали вы.

Anthropic, «Building effective agents», схема «The routing workflow».

Путь выбирает модель: агент



Действие → обратная связь от среды → снова вызов модели, пока она не решит остановиться.
Среда — это ваши инструменты. **Ровно это вы напишете сегодня вечером.**

Anthropic, «Building effective agents», схема «Autonomous agent».

Лестница автономии

СТУПЕНЬ	КТО ВЫБИРАЕТ СЛЕДУЮЩИЙ ШАГ	ГДЕ В КУРСЕ
1. Один вызов	шага нет	практика 1: заявка → JSON
2. Цепочка	вы, в коде	разбор → нормализация → запись
3. Маршрутизация	модель, из фиксированных веток	«это про план или про факт?»
4. Цикл с инструментами	модель, на каждой итерации	практика 2, сегодня
5. Автономный процесс	модель, включая момент остановки	занятие 7: фон и доставка

Что делает агент агентом

Цикл. Пока модель просит — выполнять и возвращать.

Контекст между итерациями. Каждый результат остаётся в переписке; модель видит, что уже пробовала.

Критерий остановки. Текстовый ответ вместо вызова, достигнутая цель — или `max_steps`, если ничего из этого не наступило.

Инструменты как единственный способ действовать. Всё, что агент «сделал», сделал ваш код по его просьбе. Отсюда — границы: что разрешено звать и с какими аргументами.

Вопрос 3

Где здесь агент?

1. Чат-бот поддержки, отвечает по инструкции
2. Поиск по базе знаний с генерацией ответа
3. Ваша функция `parse_order_normalized()` с прошлой практики
4. Ночной процесс: сам решает, какие витрины поднять, и пишет отчёт

Возвращаемся к вопросу 3

СИСТЕМА	АГЕНТ?	ПОЧЕМУ
Чат-бот по инструкции	нет	один вызов, действий нет
Поиск по базе знаний	нет	путь задан кодом, модель не выбирает
<code>parse_order_normalized()</code>	нет	один вызов, инструментов нет
Ночной процесс с отчётом	да	сам выбирает шаги и сам останавливается

«Сработало один раз» и «работает» — разные утверждения

pass⁸ < 25 %

t-bench: агент разговаривает с пользователем и работает с API по правилам домена. Лучшие на тот момент модели решают **меньше половины** задач.

pass^k — доля задач, решённых верно **во всех k попытках подряд**. Меньше четверти задач агент решает восемь раз из восьми.

Ваш прогон на защите — это pass¹. Поэтому на занятии 8 будет не демо, а набор регрессионных случаев.

Yao et al., «t-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains», июнь 2024.

Мост в практику

Демо и задание

Демо: один вопрос, два прогона

«Сколько вагонов выгружено на дороге ЛСН в марте?»

Прогон 1 — чат без инструментов. Ответ уверенный и с числом. Числа у модели нет: данных она не видела.

Прогон 2 — агент с инструментами. Виден весь цикл: запрос на вызов, аргументы, результат, ответ с указанием периода и фильтров.

Смотрим не только на ответы, но и на **лог цикла**: сколько итераций, какие аргументы.

Итог лекции

Инструмент — это место, где живёт точность. Модель выбирает, кого позвать, и объясняет результат. Считает — инструмент.

Агент — это цикл вокруг модели, в котором следующий инструмент она выбирает сама.

Всё, что мы будем делать оставшиеся шесть занятий, — **расширять список инструментов** и **делать цикл надёжнее**. Парадигма больше не поменяется.

Что вы напишете сегодня

Ноутбук `02_tool_calling_agent.ipynb` – второй слой вашего `logist-agent`:

- **описания трёх инструментов** над обезличенными витринами: выгрузка, погрузка, дислокация – контракт из блока 1;
- **диспетчер**: по имени из ответа модели вызвать функцию и вернуть результат – шов из блока 3;
- **цикл** с `max_steps` – блок 6;
- **системный промпт с границами**: спрашивать период, если он не назван; не выдавать текущую дислокацию за историю – четвертое место из блока 3.

Критерий сдачи – зелёный `check()`.

Что прочитать к занятию 3

Обязательно, оба текста короткие:

[Building effective agents](#) и [Writing effective tools for AI agents](#)

К практике 3, MCP:

[What is MCP](#) и [Architecture overview](#)

По интересу:

[Claude Docs, Tool use](#) · [Google, «Agents»](#) · [UC Berkeley CS294 Agentic AI](#) · [Stanford CS329Z](#)

Руками: открытые ноутбуки Anthropic, курс [tool_use](#)

И про проект

Следующее занятие — контрольная точка «Заявка на проект».

Нужно назвать: состав команды (2–3 человека), выбранную пару осей, домен и то, что считается результатом.

Комбинации осей **не повторяются** между командами и фиксируются **по порядку заявок**.

Таблица комбинаций — в syllabus.md. Девять комбинаций на все команды.